

RTAB-Map 3D Mapping and Navigation

1. Preparation

1.1 Enter the Docker environment

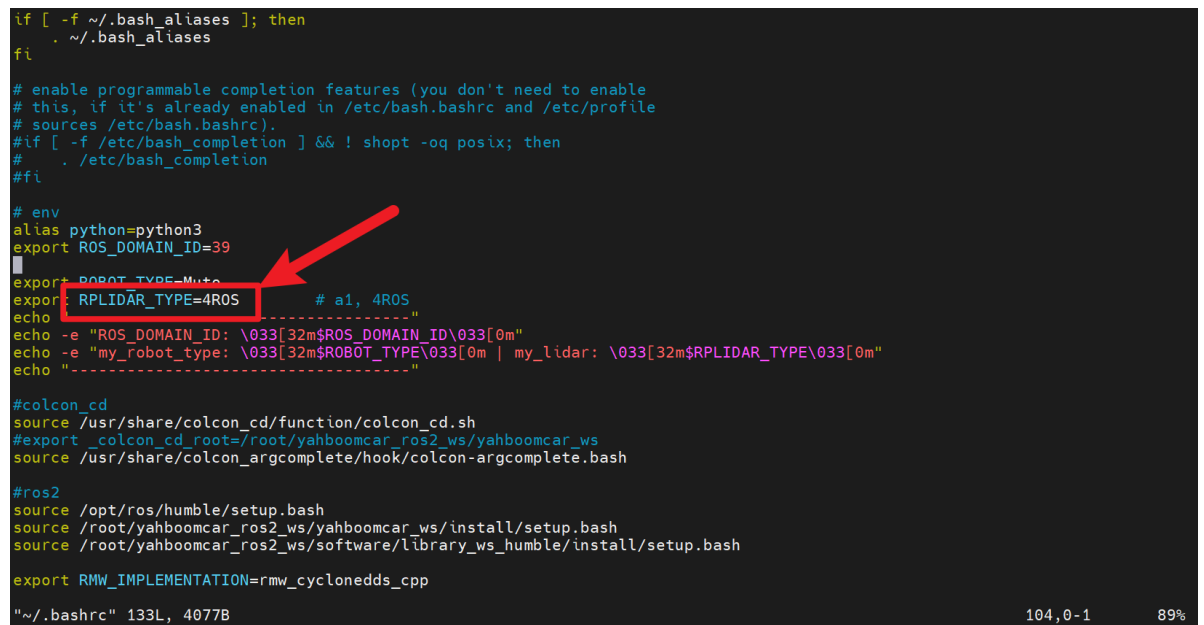
Three terminals are required for this mapping task, and two terminals are required for navigation.

After opening a new terminal, start the Docker container in one of the terminals.

```
start_muto_docker
```

In the first terminal, configure the LiDAR type.

```
vim ~/.bashrc
```



```
if [ -f ~/.bash_aliases ]; then
  . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
#if [ -f /etc/bash_completion ] && ! shopt -oq posix; then
#   . /etc/bash_completion
#fi

# env
alias python=python3
export ROS_DOMAIN_ID=39
export ROBOT_TYPE=Muto
export RPLIDAR_TYPE=4ROS # a1, 4ROS
echo "-----"
echo -e "ROS_DOMAIN_ID: \033[32m$ROS_DOMAIN_ID\033[0m"
echo -e "my_robot_type: \033[32m$ROBOT_TYPE\033[0m | my_lidar: \033[32m$RPLIDAR_TYPE\033[0m"
echo "-----"

#colcon_cd
source /usr/share/colcon_cd/function/colcon_cd.sh
#export _colcon_cd_root=/root/yahboomcar_ros2_ws/yahboomcar_ws
source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash

#ros2
source /opt/ros/humble/setup.bash
source /root/yahboomcar_ros2_ws/yahboomcar_ws/install/setup.bash
source /root/yahboomcar_ros2_ws/software/library_ws_humble/install/setup.bash

export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

"~/.bashrc" 133L, 4077B 104,0-1 89%
```

The other terminals only need to enter this already started container.

```
enter_muto_docker
```

```

pi@raspberrypi:~$ enter_muto_docker
正在进入容器 dd6b110e456c 的终端...
-----
ROS_DOMAIN_ID: 39
my_robot_type: Muto | my_lidar: 4R0S
-----
root@raspberrypi:/#

```

2. Mapping (RTAB-Map Mapping)

Target Deliverables

Ultimately, we need to save two types of data from mapping:

- **RTAB-Map Database:** `rtabmap.db` (for 3D map/playback/visualization/subsequent localization)
- **Grid Map for Navigation:** `rtab_map.yaml` + `rtab_map.pgm` (for use with Nav2's `map_server`)

2.1 Step 1: Start Mapping

```
ros2 launch yahboomcar_nav map_rtabmap_complete_launch.py
```

This command will start the "camera + mapping-related nodes (odometry)" together.

```

[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 53.773375
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.034136 -0.000641 0.001307]
[rft2_laser_odometry_node-2] BASEodom = [-0.002492 -0.000000 0.001307]
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 45.868546
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.034204 -0.000669 0.000616]
[rft2_laser_odometry_node-2] BASEodom = [-0.002501 -0.000649 0.000616]
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 48.138346
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.034421 -0.001062 0.000128]
[rft2_laser_odometry_node-2] BASEodom = [-0.002778 -0.001058 0.000128]
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 51.562945
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.034110 -0.000858 0.000726]
[rft2_laser_odometry_node-2] BASEodom = [-0.002466 -0.000827 0.000726]
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 46.328471
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.034101 -0.000949 0.000726]
[rft2_laser_odometry_node-2] BASEodom = [-0.002547 -0.000926 0.000726]
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 57.747435
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.033914 -0.001056 -0.000251]
[rft2_laser_odometry_node-2] BASEodom = [-0.002270 -0.001057 -0.000251]
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 56.254932
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.033746 -0.001013 0.000440]
[rft2_laser_odometry_node-2] BASEodom = [-0.002102 -0.000999 0.000440]
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 44.715439
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.033955 -0.001156 0.000785]
[rft2_laser_odometry_node-2] BASEodom = [-0.002312 -0.001131 0.000785]
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 52.095892
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.033863 -0.001414 0.000785]
[rft2_laser_odometry_node-2] BASEodom = [-0.002210 -0.001389 0.000785]
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 43.046261
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.033877 -0.001157 0.000785]
[rft2_laser_odometry_node-2] BASEodom = [-0.002234 -0.001132 0.000785]
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 50.700256
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.034063 -0.001130 -0.000360]
[rft2_laser_odometry_node-2] BASEodom = [-0.002410 -0.001142 -0.000360]
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 44.616819
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.034011 -0.001132 0.000553]
[rft2_laser_odometry_node-2] BASEodom = [-0.002368 -0.001295 0.000553]
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] Waiting for laser_scans....
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 45.350598
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.034191 -0.001056 -0.000360]
[rft2_laser_odometry_node-2] BASEodom = [-0.002547 -0.001068 -0.000360]
[rft2_laser_odometry_node-2] [rft2o] execution time (ms): 45.564376
[rft2_laser_odometry_node-2] [rft2o] LASERodom = [-0.034238 -0.001264 0.000553]
[rft2_laser_odometry_node-2] BASEodom = [-0.002594 -0.001246 0.000553]
[rft2_laser_odometry_node-2] Waiting for laser_scans....

```

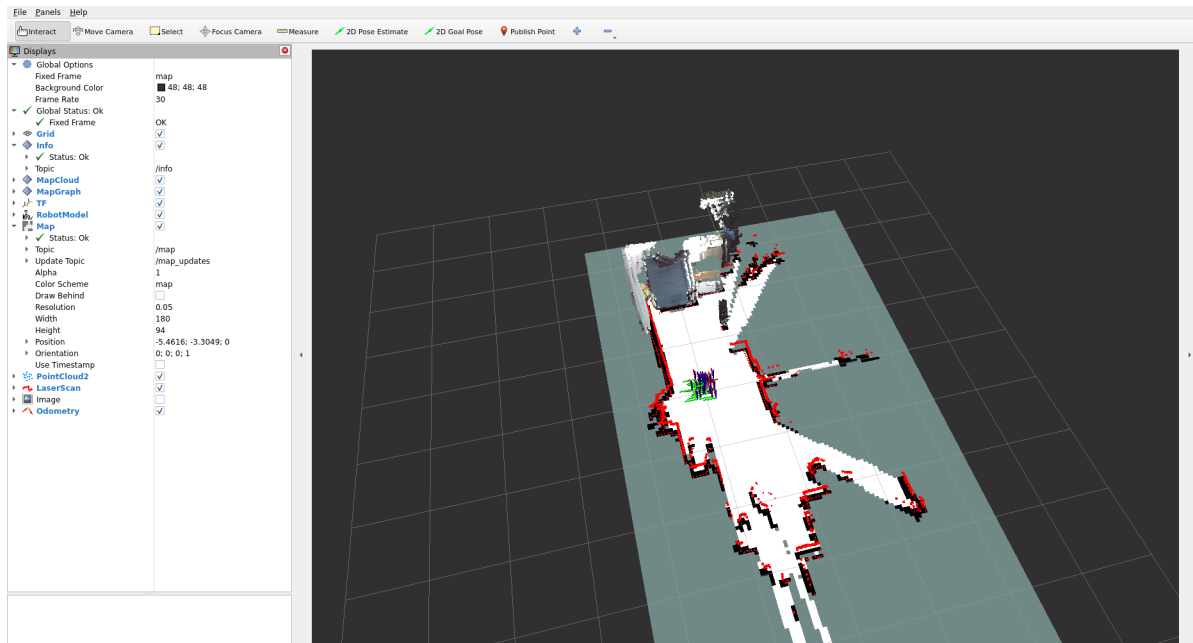
2.2 Step 2: Open RViz to view the mapping process

In another terminal, enter Docker (execute enter_muto_docker).

This step can also be performed in virtual machine, where RViz will run much more smoothly.

```
ros2 launch yahboomcar_nav display_rtabmap_map_launch.py
```

- RViz's Fixed Frame is generally set to `map`.
- If you can see the map/point cloud display from RTAB-Map, it means the mapping link is basically established.



2.3 Step 3: Use keyboard to control the mobile robot to collect data

```
ros2 run yahboomcar_ctrl yahboom_keyboard
```

```
root@raspberrypi:/# ros2 run yahboomcar_ctrl yahboom_keyboard
```

```
Control Your SLAM-Bot!
```

```
-----  
Moving around:
```

```
u   i   o  
j   k   l  
m   ,   .
```

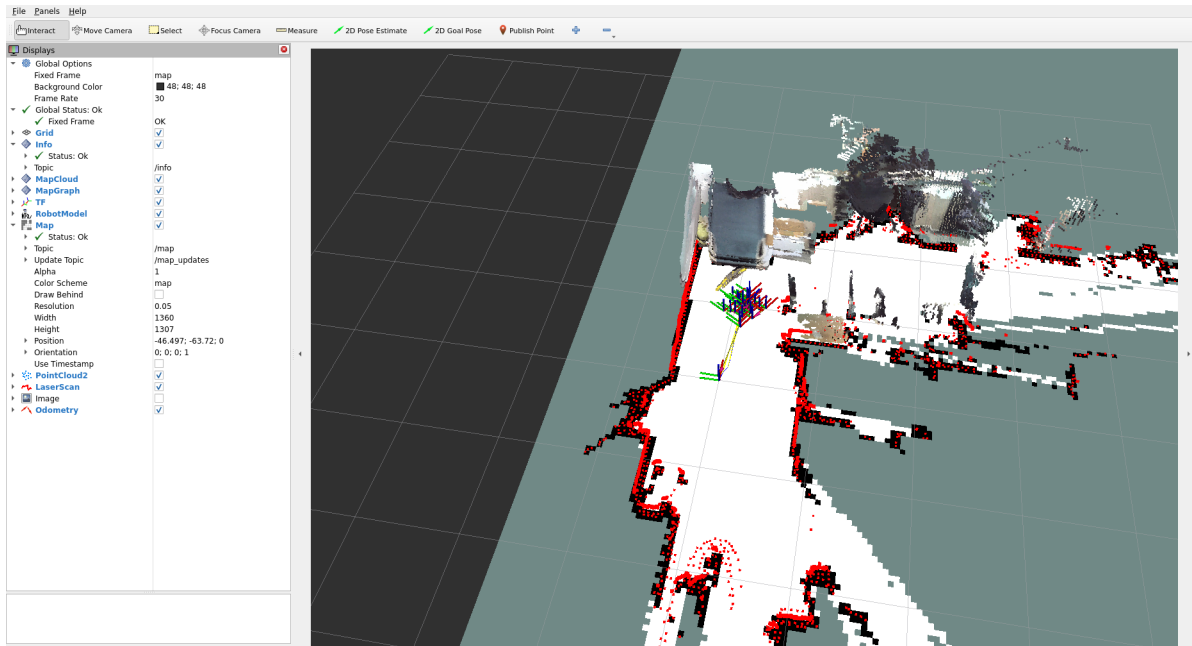
```
q/z : increase/decrease max speeds by 10%  
w/x : increase/decrease only linear speed by 10%  
e/c : increase/decrease only angular speed by 10%  
t/T : x and y speed switch  
s/S : stop keyboard control  
space key, k : force stop  
anything else : stop smoothly
```

```
CTRL-C to quit
```

```
currently:      speed 0.2      turn 1.0
```

Move Muto according to the key prompts. Suggestions:

- Move slowly and at a constant speed.
- Try to perform a "closed loop" (return to near the starting point).
- Avoid rapid rotation/shaking, as the depth camera can easily lose features.



2.4 Save the map (a crucial step after mapping is complete)

Return to the navigation terminal and press Ctrl+C to end the process, which will save the map's db data.

```
[camera_info_republisher-7] Traceback (most recent call last):
[camera_info_republisher-7]   File "/root/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_nav/lib/python3.10/site-packages/yahboomcar_nav/camera_info_republisher.py", line 149,
[camera_info_republisher-7]     rclpy.spin(node)
[camera_info_republisher-7]   File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/__init__.py", line 222, in spin
[camera_info_republisher-7]     executor.spin_once()
[camera_info_republisher-7]   File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 739, in spin_once
[camera_info_republisher-7]     self.spin_once_impl(timeout_sec)
[camera_info_republisher-7]   File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 728, in _spin_once_impl
[camera_info_republisher-7]     handler, entity = self.wait_for_ready_callbacks(timeout_sec=timeout_sec)
[camera_info_republisher-7]   File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 711, in wait_for_ready_callbacks
[camera_info_republisher-7]     return next(self._cb_iter)
[camera_info_republisher-7]   File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 688, in _wait_for_ready_callbacks
[camera_info_republisher-7]     wait_set.wait(timeout_nsec)
[camera_info_republisher-7] KeyboardInterrupt
[camera_info_republisher-7] During handling of the above exception, another exception occurred:
[camera_info_republisher-7] Traceback (most recent call last):
[camera_info_republisher-7]   File "/root/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_nav/lib/yahboomcar_nav/camera_info_republisher", line 33, in <module>
[camera_info_republisher-7]     sys.exit(load_entry_point('yahboomcar-nav==0.0.0', 'console_scripts', 'camera_info_republisher')())
[camera_info_republisher-7]   File "/root/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_nav/lib/python3.10/site-packages/yahboomcar_nav/camera_info_republisher.py", line 152,
[camera_info_republisher-7]     rclpy.shutdown()
[camera_info_republisher-7]   File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/__init__.py", line 126, in shutdown
[camera_info_republisher-7]     shutdown(context=context)
[camera_info_republisher-7]   File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/utilities.py", line 58, in shutdown
[camera_info_republisher-7]     return context.shutdown()
[camera_info_republisher-7]   File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/context.py", line 102, in shutdown
[camera_info_republisher-7]     self._context.shutdown()
[camera_info_republisher-7] rclpy.rclpy_pybind11.RCLError: failed to shutdown: rcl_shutdown already called on the given context, at ./src/rcl/init.c:241
[INFO] [joint_state_publisher-3]: process has finished cleanly [pid 125]
[ERROR] [camera_info_republisher-7]: process has died [pid 133, exit code 1, cmd '/root/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_nav/lib/yahboomcar_nav/camera_info_repub
[INFO] [rgbd_sync-8]: process has finished cleanly [pid 135]
[ERROR] [muto_driver-5]: process has died [pid 129, exit code -2, cmd '/root/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_bringup/lib/yahboomcar_bringup/muto_driver --ros-ar
gs'].
[INFO] [astra_camera_node-1]: process has finished cleanly [pid 121]
[rtabmap-9] [WARN] (2026-01-09 11:12:41.385) MapsManager.cpp:863:publishMaps() Graph has changed! The whole cloud is regenerated.
[rtabmap-9] rtabmap: Saving database/long-term memory... (located at /root/.ros/rtabmap.db)
[rtabmap-9] rtabmap: 2D occupancy grid map saved.
[rtabmap-9] rtabmap: Saving database/long-term memory...done! (located at /root/.ros/rtabmap.db, 77 MB)

[ERROR] [ydlidar_ros2_driver_node-6]: process[ydlidar_ros2_driver_node-6] failed to terminate '5' seconds after receiving 'SIGINT', escalating to 'SIGTERM'
[INFO] [ydlidar_ros2_driver_node-6]: sending signal 'SIGTERM' to process[ydlidar_ros2_driver_node-6]
[ERROR] [ydlidar_ros2_driver_node-6]: process has died [pid 131, exit code -15, cmd '/root/yahboomcar_ros2_ws/software/library_ws_humble/install/ydlidar_ros2_driver/lib/ydlidar_ros
2_driver/ydlidar_ros2_driver_node --ros-args -r __node:=ydlidar_ros2_driver_node --params-file /tmp/launch_params_26nz1szf'].
root@raspberrypi:~#
```

Back up the RTAB-Map database (optional)

Each time you start mapping, the RTAB-Map database will be overwritten.

- `/root/.ros/rtabmap.db`

Back it up to a fixed location:

```
cp /root/.ros/rtabmap.db ~/yahboomcar_ros2_ws/maps/rtabmap.db
```

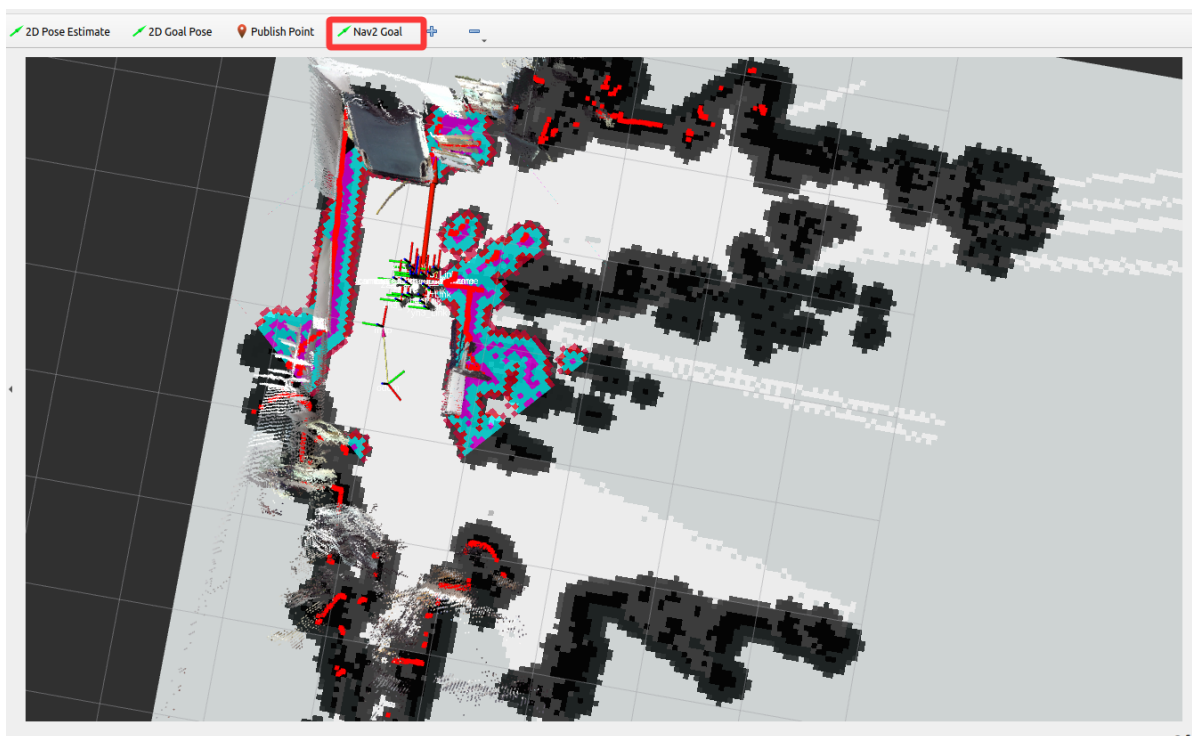
3. Navigation (AMCL + Map Server + Nav2)

3.1 One-click start

```
ros2 launch yahboomcar_nav rtabmap_nav_launch.py with_rtabmap:=true  
with_odom:=true launch_camera:=true publish_tf:=true
```

3.2 Step 2: Open navigation RViz (recommended to start in virtual machine)

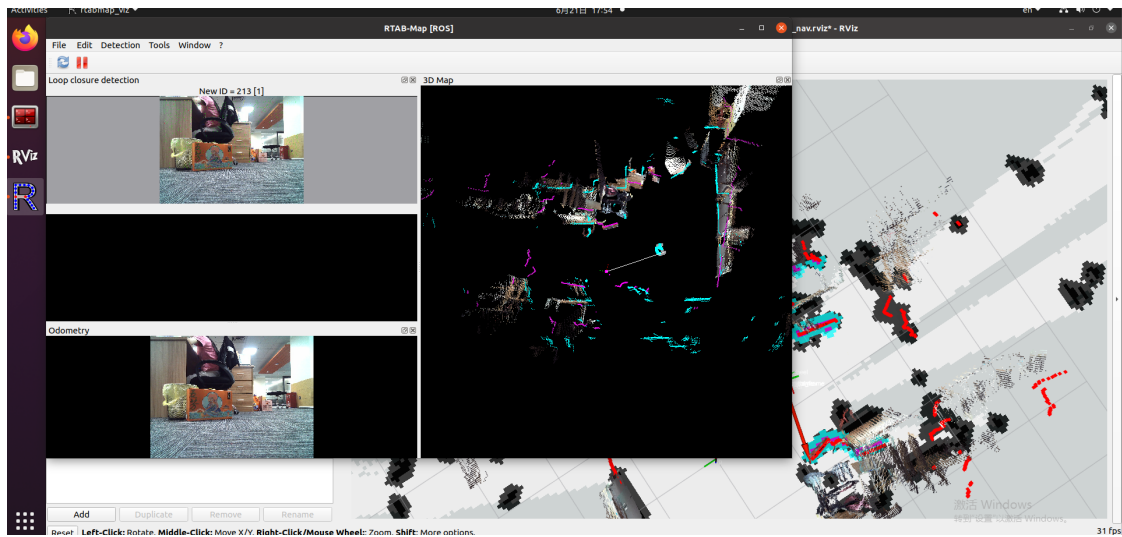
```
ros2 launch yahboomcar_nav display_rtabmap_nav_launch.py
```



Here we can also open an `rtabmap_viz` window. `rtabmap_viz` is the visualization interface for RTAB-Map, a wrapper for the RTAB-Map GUI graphics library. It is similar to `rviz` but has options specific to RTAB-Map. It can subscribe to different topics, such as `odom`, `rgb/image`, `depth/image`, `scan`, etc., to display the SLAM process and results, while loading the 3D map into `rviz`. It is recommended to start this step in a virtual machine.

Note that after starting this step, the map will not be displayed. You need to wait for the navigation node to start in the next step before it will be displayed.

```
ros2 launch yahboomcar_nav rtabmap_viz_launch.py
```



3.3 Standard navigation operations in RViz

1. Set Fixed Frame to `map`.
2. Click `Nav2 Goal`, select a target location on the map, and drag a directional arrow.
3. The robot starts planning and moving.